

ITA06 - Machine Learning

Name: Abdul Azeez S

Reg: 192413012

EXP NO:1

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

AIM

To implement the Find-S algorithm to find the most specific hypothesis from the given training data.

ALGORITHM

Initialize the hypothesis h with the most specific values (0).

1. Read the training examples.
2. Consider only positive examples (Yes).
3. Compare each attribute with the hypothesis.
4. If values are different, replace with ?.
5. Repeat for all training examples.
6. Display the final hypothesis.

PROGRAM CODE:

Find-S Algorithm Implementation

```
import pandas as pd
```

```
# Training dataset
```

```
data = {  
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],  
    'Temp': ['Warm', 'Warm', 'Cold', 'Warm'],
```

```
'Humidity': ['Normal', 'High', 'High', 'High'],  
'Wind': ['Strong', 'Strong', 'Strong', 'Strong'],  
'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']  
}
```

```
df = pd.DataFrame(data)
```

```
print("Training Data:")
```

```
print(df)
```

```
# Separate attributes and target
```

```
attributes = df.iloc[:, :-1].values
```

```
target = df.iloc[:, -1].values
```

```
# Initialize hypothesis
```

```
hypothesis = attributes[0].copy()
```

```
print("\nInitial Hypothesis:", hypothesis)
```

```
# Apply Find-S algorithm
```

```
for i in range(len(attributes)):
```

```
    if target[i] == "Yes":
```

```
        for j in range(len(hypothesis)):
```

```
            if hypothesis[j] != attributes[i][j]:
```

```
                hypothesis[j] = '?'
```

```
print("\nFinal Most Specific Hypothesis:")
```

```
print(hypothesis)
```

OUTPUT:

```
if hypothesis[j] != attributes[i][j]:
    hypothesis[j] = '?'

print("\nFinal Most Specific Hypothesis:")
print(hypothesis)
```

... Training Data:

	Sky	Temp	Humidity	Wind	EnjoySport
0	Sunny	Warm	Normal	Strong	Yes
1	Sunny	Warm	High	Strong	Yes
2	Rainy	Cold	High	Strong	No
3	Sunny	Warm	High	Strong	Yes

Initial Hypothesis: ['Sunny' 'Warm' 'Normal' 'Strong']

Final Most Specific Hypothesis:
['Sunny' 'Warm' '?' 'Strong']

RESULT

Thus, the Find-S algorithm was successfully implemented using Python, and the most specific hypothesis consistent with the given training data was obtained.

EXP NO:2

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

AIM

To implement the Find-S algorithm to find the most specific hypothesis from the given training data using Python.

ALGORITHM

1. Initialize the hypothesis h with the most specific values (0).
2. Read the training dataset.
3. Select the first positive example.
4. Compare the hypothesis with each positive training example.
5. If an attribute value differs, replace it with ?.

6. Ignore negative examples.
7. Repeat the process until all examples are processed.
8. Display the final hypothesis.

PROGRAM

Find-S Algorithm

```
data = [  
    ['Sunny','Warm','Normal','Strong','Yes'],  
    ['Sunny','Warm','High','Strong','Yes'],  
    ['Rainy','Cold','High','Strong','No'],  
    ['Sunny','Warm','High','Strong','Yes']  
]
```

number of attributes

```
n = len(data[0]) - 1
```

initialize hypothesis

```
hypothesis = ['0'] * n
```

for sample in data:

```
    if sample[-1] == "Yes":
```

```
        for i in range(n):
```

```
            if hypothesis[i] == '0':
```

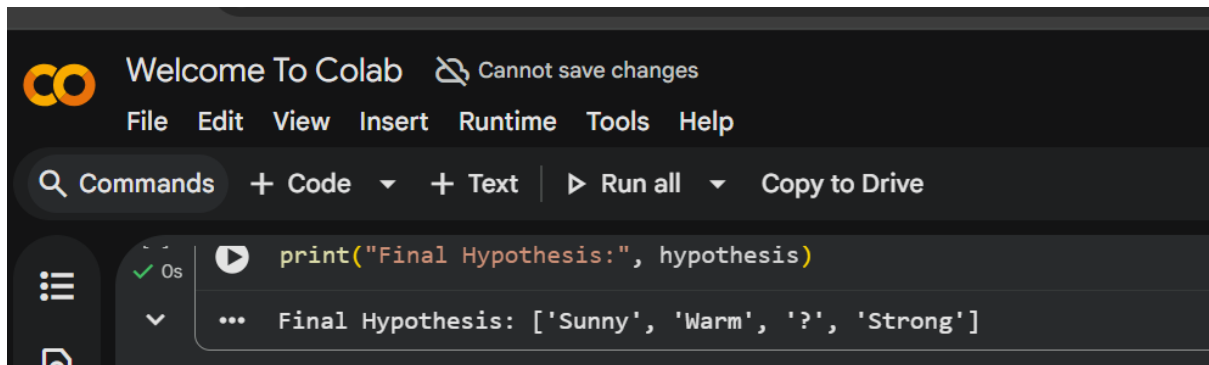
```
                hypothesis[i] = sample[i]
```

```
            elif hypothesis[i] != sample[i]:
```

```
                hypothesis[i] = '?'
```

```
print("Final Hypothesis:", hypothesis)
```

OUTPUT:



The screenshot shows the Google Colab interface. At the top, there's a 'Welcome To Colab' banner with a 'Cannot save changes' warning. Below the banner is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A toolbar contains a search icon, 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. A code cell is visible with the code `print("Final Hypothesis:", hypothesis)`. The output of the cell is displayed below the code: `Final Hypothesis: ['Sunny', 'Warm', '?', 'Strong']`.

RESULT

Thus, the Find-S algorithm was successfully implemented and the most specific hypothesis consistent with the given training examples was obtained.

EXP NO:3

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

AIM

To implement a Decision Tree using the ID3 algorithm and classify a new sample.

ALGORITHM

1. Calculate entropy of the dataset.
2. Compute information gain for each attribute.
3. Choose the attribute with highest information gain as root.
4. Split dataset based on that attribute.
5. Repeat until all samples are classified.
6. Use the tree to classify new data.

CODE

```
from sklearn.tree import DecisionTreeClassifier  
  
from sklearn.datasets import load_iris  
  
from sklearn.model_selection import train_test_split
```

```
data = load_iris()
```

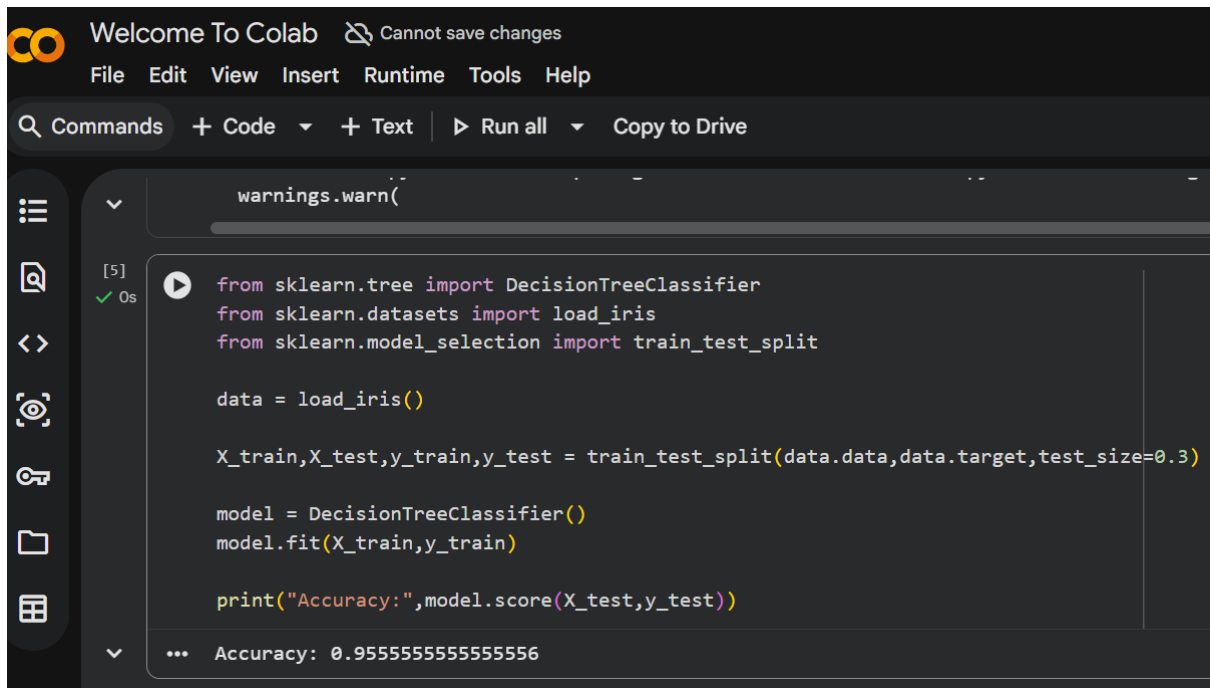
```
X_train,X_test,y_train,y_test = train_test_split(data.data,data.target,test_size=0.3)
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X_train,y_train)
```

```
print("Accuracy:",model.score(X_test,y_test))
```

OUTPUT:



```
Welcome To Colab  Cannot save changes
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text | ▶ Run all Copy to Drive

[5]
✓ 0s
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()

X_train,X_test,y_train,y_test = train_test_split(data.data,data.target,test_size=0.3)

model = DecisionTreeClassifier()
model.fit(X_train,y_train)

print("Accuracy:",model.score(X_test,y_test))

... Accuracy: 0.9555555555555556
```

RESULT

Thus the Decision Tree using ID3 algorithm was successfully implemented and classification was performed.

EXP NO:4

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

AIM

To build an Artificial Neural Network using the Backpropagation algorithm.

ALGORITHM

1. Initialize weights randomly.
2. Provide input data.
3. Perform forward propagation.
4. Compute error between predicted and actual output.
5. Update weights using backpropagation.
6. Repeat until minimum error is obtained.

CODE

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

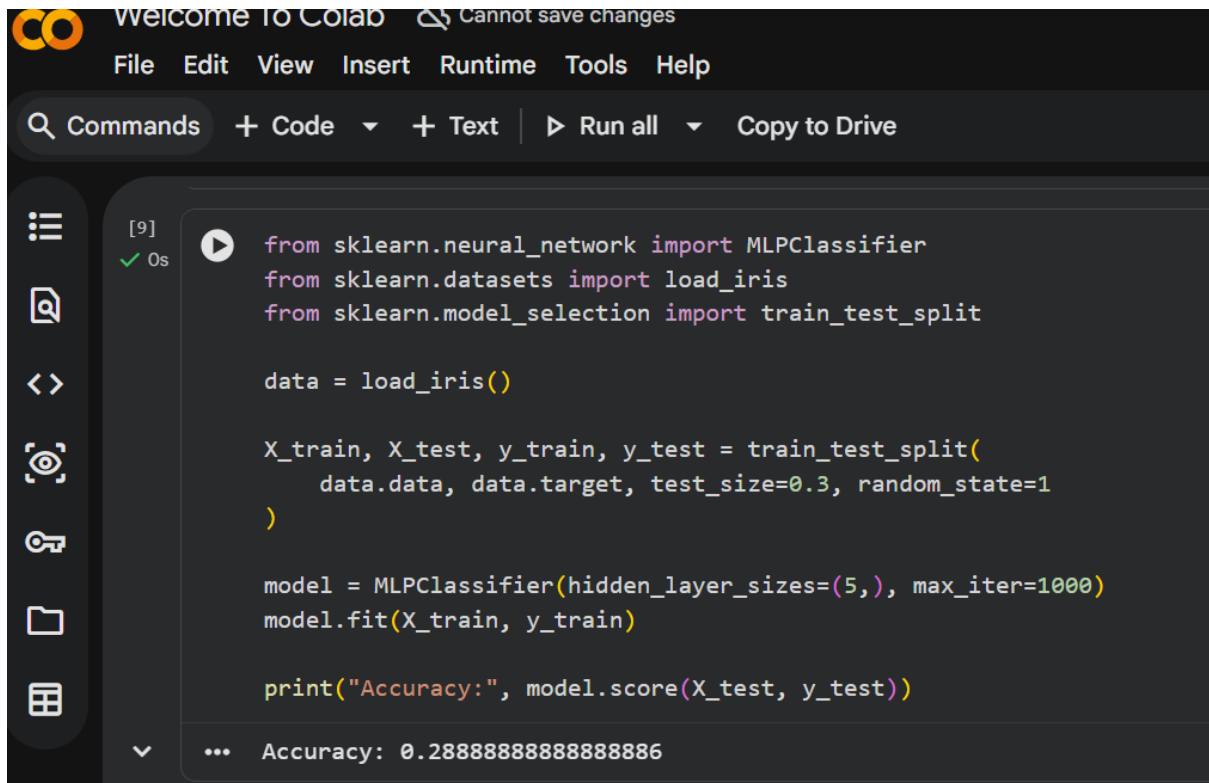
data = load_iris()

X_train,X_test,y_train,y_test = train_test_split(data.data,data.target,test_size=0.3)

model = MLPClassifier(hidden_layer_sizes=(5,),max_iter=1000)
model.fit(X_train,y_train)

print("Accuracy:",model.score(X_test,y_test))
```

OUTPUT:



```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=1
)

model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000)
model.fit(X_train, y_train)

print("Accuracy:", model.score(X_test, y_test))
```

... Accuracy: 0.28888888888888886

RESULT

Thus the Artificial Neural Network using Backpropagation was successfully implemented.

EXP NO:5

Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

AIM

To implement the K-Nearest Neighbors algorithm.

ALGORITHM

1. Select the value of K.
2. Calculate distance between test and training samples.
3. Choose K nearest neighbors.
4. Find majority class.
5. Assign the class to test data.

CODE

```
from sklearn.neighbors import KNeighborsClassifier

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split


data = load_iris()

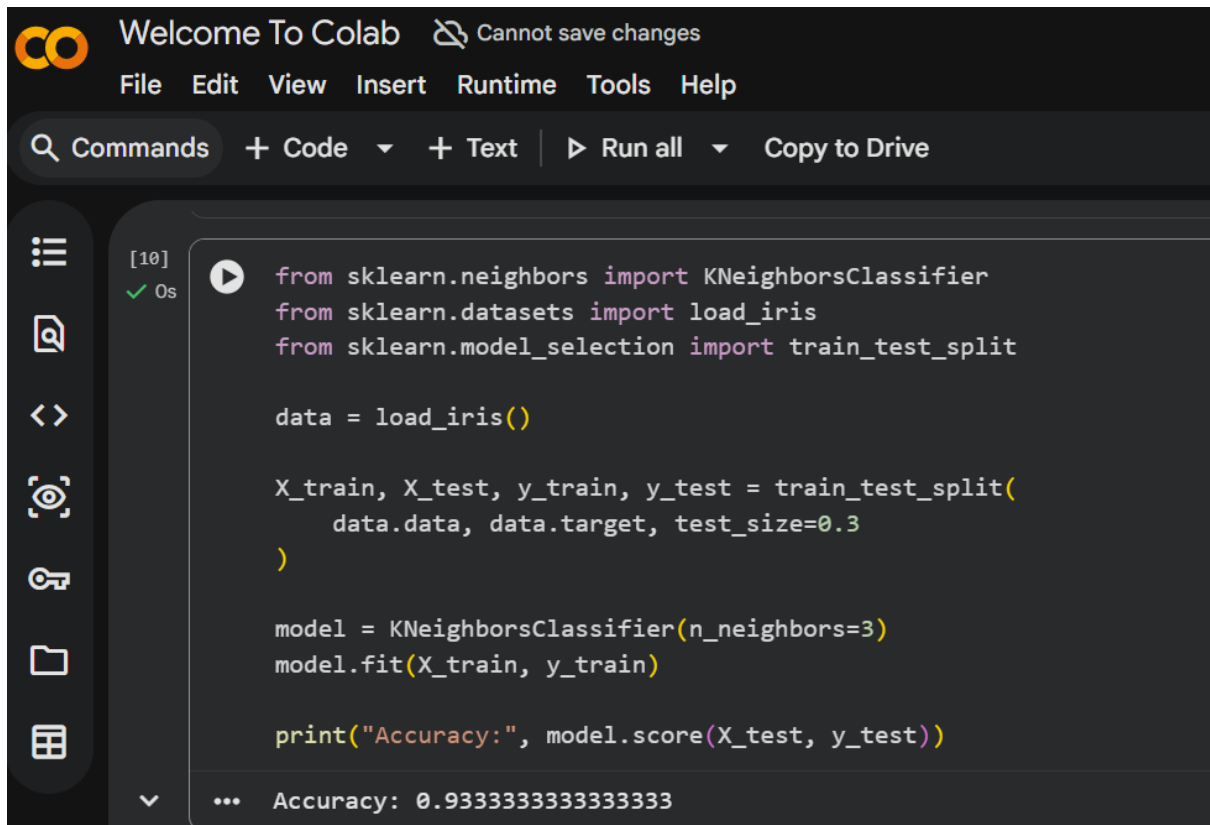

X_train,X_test,y_train,y_test = train_test_split(data.data,data.target,test_size=0.3)


model = KNeighborsClassifier(n_neighbors=3)

model.fit(X_train,y_train)


print("Accuracy:",model.score(X_test,y_test))
```

OUTPUT:

A screenshot of a Google Colab notebook interface. The top bar shows the Colab logo, 'Welcome To Colab', and a warning 'Cannot save changes'. Below this is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A toolbar contains a search icon, 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. On the left is a sidebar with icons for file explorer, search, and other functions. The main area shows a code cell with the following Python code:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

print("Accuracy:", model.score(X_test, y_test))
```

 The cell is marked as '[10] ✓ 0s'. Below the code, the output is displayed: '... Accuracy: 0.9333333333333333'.

RESULT

Thus the K-Nearest Neighbours algorithm was successfully implemented.

EXP NO:6

Write a program to implement Naive Bayes algorithm in python and to display the results using confusion matrix and accuracy.

AIM

To implement the Naive Bayes classifier and evaluate using confusion matrix and accuracy.

ALGORITHM

1. Read training dataset.
2. Calculate prior probability of each class.
3. Calculate likelihood probabilities.
4. Apply Bayes theorem.
5. Predict class with highest probability.

CODE

```
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix,accuracy_score

data = load_iris()

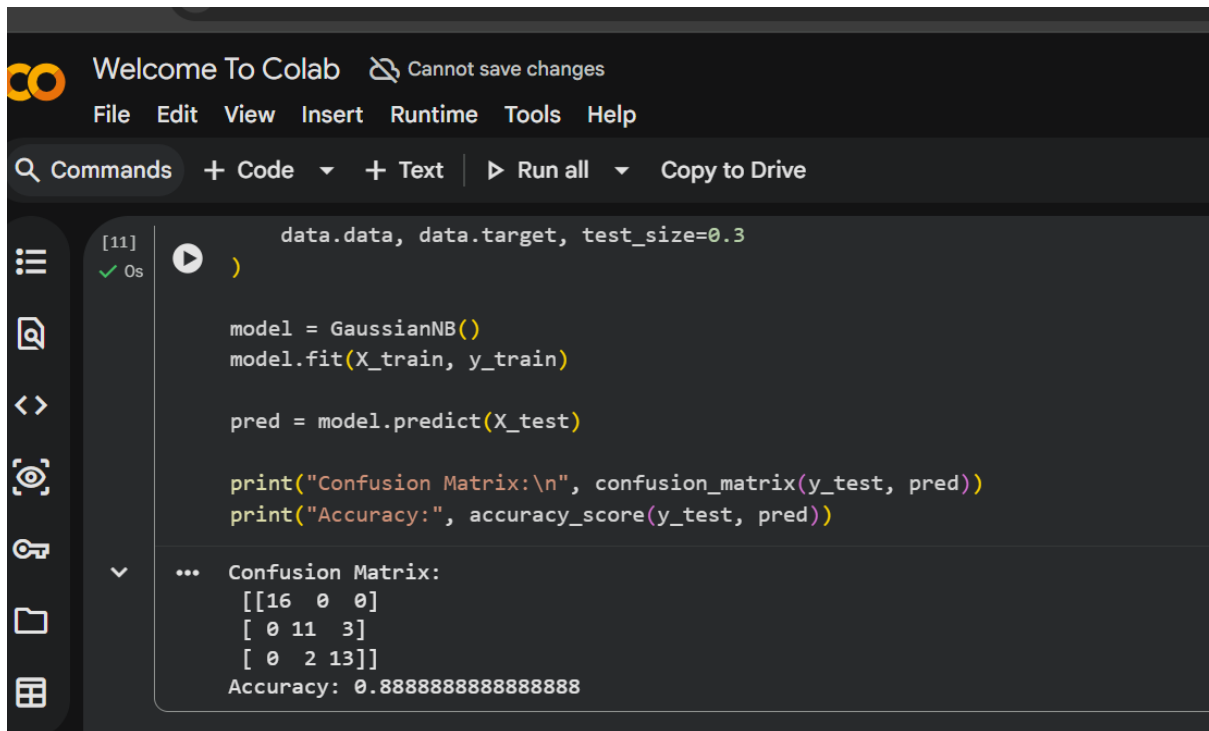
X_train,X_test,y_train,y_test = train_test_split(data.data,data.target,test_size=0.3)

model = GaussianNB()
model.fit(X_train,y_train)

pred = model.predict(X_test)

print("Confusion Matrix:\n",confusion_matrix(y_test,pred))
print("Accuracy:",accuracy_score(y_test,pred))
```

OUTPUT:



The screenshot shows a Google Colab interface with a dark theme. At the top, it says 'Welcome To Colab' and 'Cannot save changes'. Below the menu bar (File, Edit, View, Insert, Runtime, Tools, Help), there's a toolbar with 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. The main area contains a code cell with the following Python code:

```
[11] data.data, data.target, test_size=0.3
)

model = GaussianNB()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
print("Accuracy:", accuracy_score(y_test, pred))
```

The output of the code cell is displayed below the code:

```
... Confusion Matrix:
[[16  0  0]
 [ 0 11  3]
 [ 0  2 13]]
Accuracy: 0.8888888888888888
```

RESULT

Thus the Naïve Bayes algorithm was successfully implemented and evaluated.

EXP NO:7

Write a program to implement Logistic Regression (LR) algorithm in python

AIM

To implement Logistic Regression.

ALGORITHM

1. Initialize model parameters.
2. Apply sigmoid function to compute probability.
3. Calculate error.
4. Update weights using gradient descent.
5. Repeat until convergence.

CODE

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```
data = load_iris()
```

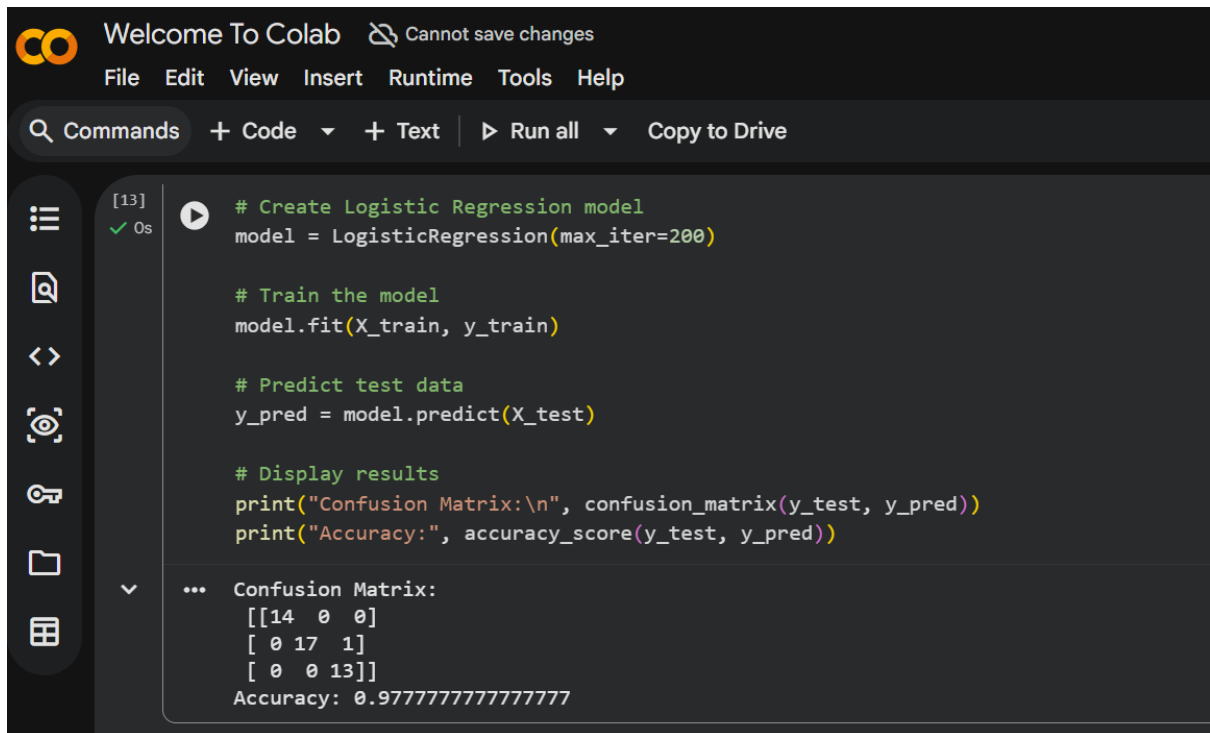
```
X_train,X_test,y_train,y_test = train_test_split(data.data,data.target,test_size=0.3)
```

```
model = LogisticRegression(max_iter=200)
```

```
model.fit(X_train,y_train)
```

```
print("Accuracy:",model.score(X_test,y_test))
```

OUTPUT:



```
Welcome To Colab  Cannot save changes
File Edit View Insert Runtime Tools Help

Commands + Code + Text | Run all Copy to Drive

[13] ✓ 0s # Create Logistic Regression model
model = LogisticRegression(max_iter=200)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Display results
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))

... Confusion Matrix:
[[14  0  0]
 [ 0 17  1]
 [ 0  0 13]]
Accuracy: 0.9777777777777777
```

RESULT

Thus the Logistic Regression algorithm was successfully implemented.

EXP NO:8

Write a program to implement Linear Regression (LR) algorithm in python

AIM

To implement Linear Regression.

ALGORITHM

1. Read dataset.
2. Define input and output variables.
3. Fit the regression model.
4. Compute regression line.
5. Predict output.

CODE

Linear Regression in Python

```
from sklearn.linear_model import LinearRegression
import numpy as np
```

```
# Sample dataset
```

```
X = np.array([[1], [2], [3], [4], [5]]) # Independent variable
```

```
y = np.array([2, 4, 6, 8, 10]) # Dependent variable
```

```
# Create Linear Regression model
```

```
model = LinearRegression()
```

```
# Train the model
```

```
model.fit(X, y)
```

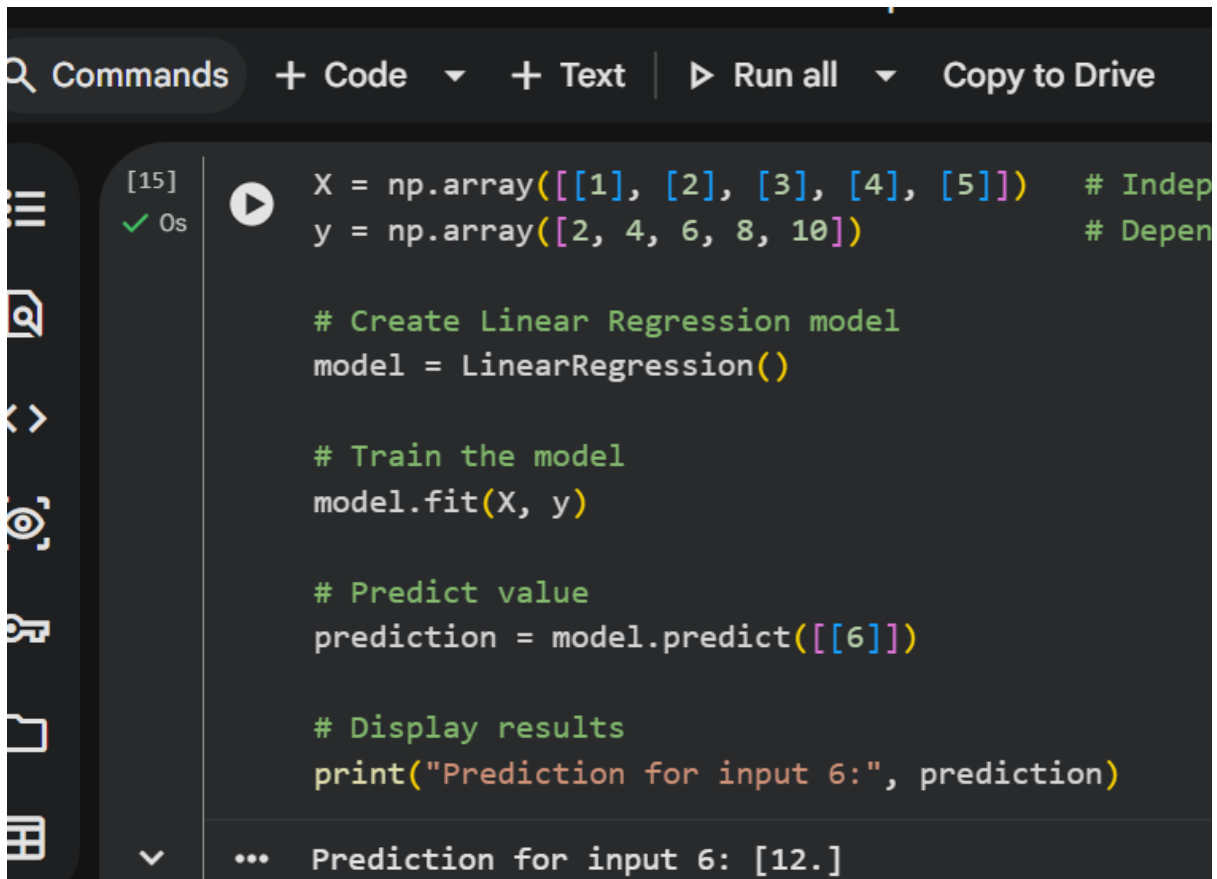
```
# Predict value
```

```
prediction = model.predict([[6]])
```

```
# Display results
```

```
print("Prediction for input 6:", prediction)
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface with a dark theme. At the top, there is a toolbar with buttons for 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. On the left, a sidebar contains icons for file management and search. The main area displays a code cell with the following Python code:

```
[15] X = np.array([[1], [2], [3], [4], [5]]) # Indep
      y = np.array([2, 4, 6, 8, 10]) # Depen

      # Create Linear Regression model
      model = LinearRegression()

      # Train the model
      model.fit(X, y)

      # Predict value
      prediction = model.predict([[6]])

      # Display results
      print("Prediction for input 6:", prediction)
```

Below the code, the output is displayed: "... Prediction for input 6: [12.]".

RESULT

Thus the Linear Regression model was successfully implemented.

EXP NO:9

Compare Linear and Polynomial Regression using Python

AIM

To compare Linear Regression and Polynomial Regression.

ALGORITHM

1. Load dataset.
2. Fit linear regression model.
3. Convert features into polynomial form.
4. Fit polynomial regression model.

5. Compare results.

CODE

```
import numpy as np  
from sklearn.linear_model import LinearRegression  
from sklearn.preprocessing import PolynomialFeatures
```

```
X = np.array([1,2,3,4,5]).reshape(-1,1)
```

```
y = np.array([1,4,9,16,25])
```

```
lin = LinearRegression()
```

```
lin.fit(X,y)
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X)
```

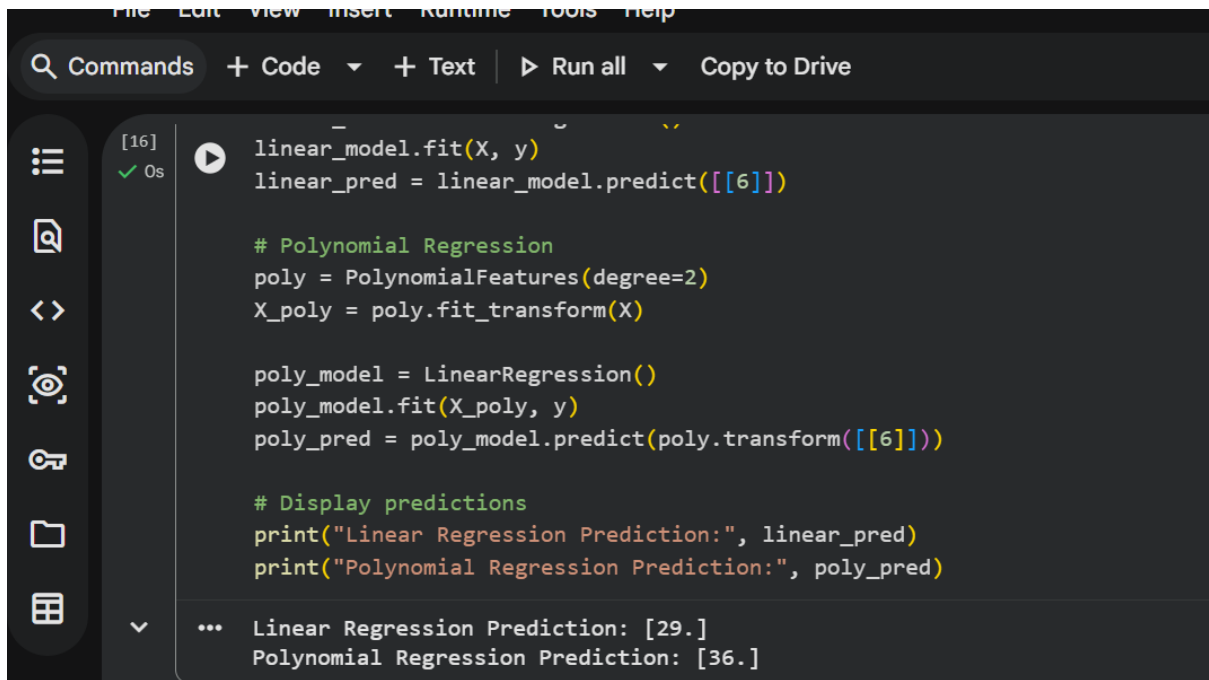
```
lin2 = LinearRegression()
```

```
lin2.fit(X_poly,y)
```

```
print("Linear:",lin.predict([[6]]))
```

```
print("Polynomial:",lin2.predict(poly.transform([[6]])))
```

OUTPUT:



```
linear_model.fit(X, y)
linear_pred = linear_model.predict([[6]])

# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

poly_model = LinearRegression()
poly_model.fit(X_poly, y)
poly_pred = poly_model.predict(poly.transform([[6]]))

# Display predictions
print("Linear Regression Prediction:", linear_pred)
print("Polynomial Regression Prediction:", poly_pred)
```

... Linear Regression Prediction: [29.]
Polynomial Regression Prediction: [36.]

RESULT

Thus the comparison between linear and polynomial regression was successfully performed.

EXP NO:10

Write a Python Program to Implement Expectation & Maximization Algorithm

AIM

To implement the Expectation-Maximization algorithm.

ALGORITHM

1. Initialize parameters randomly.
2. Perform Expectation step to estimate probabilities.
3. Perform Maximization step to update parameters.
4. Repeat until convergence.
5. Assign clusters.

CODE

```
from sklearn.mixture import GaussianMixture
```

```
import numpy as np
```

```
X = np.array([[1],[2],[3],[10],[11],[12]])
```

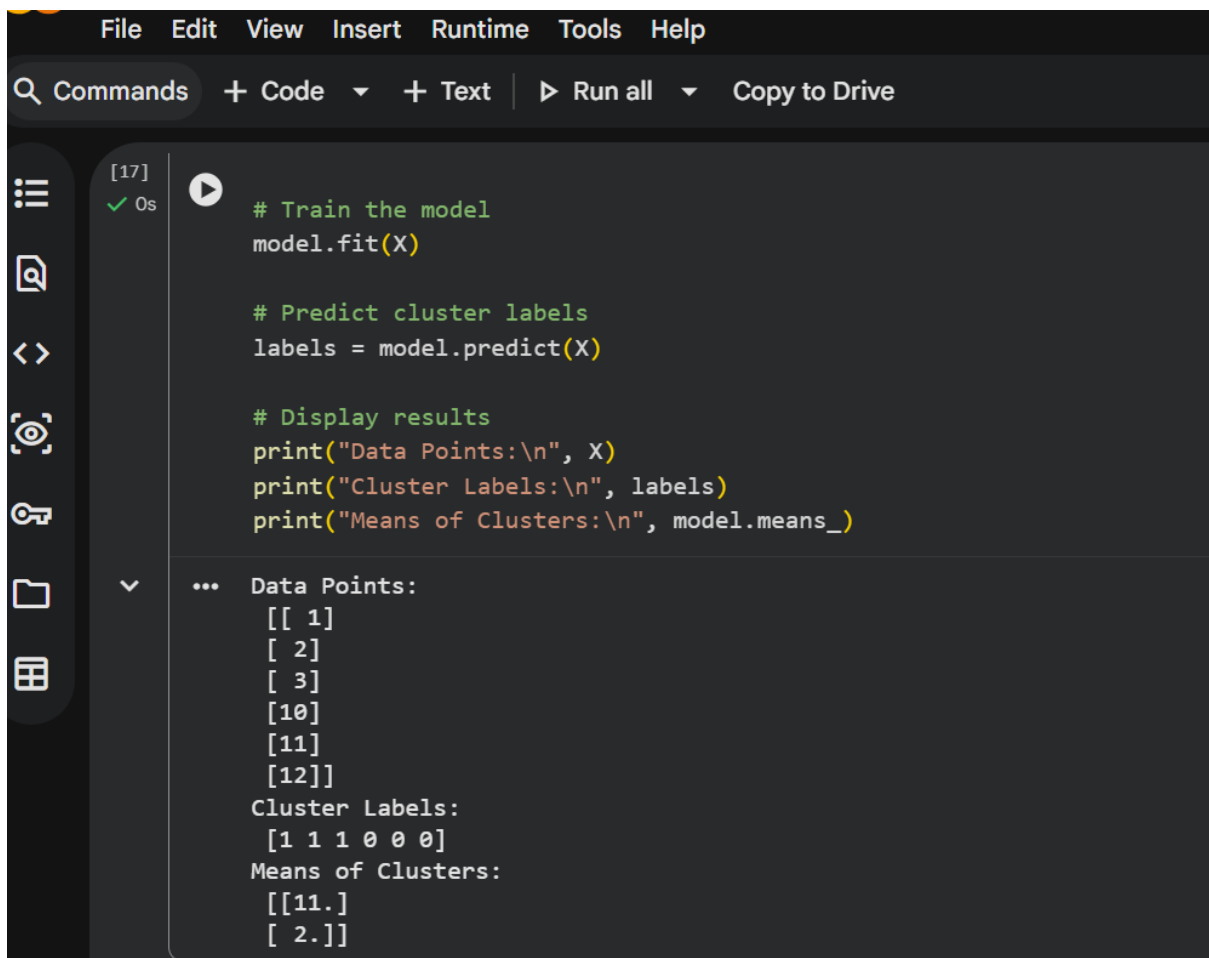
```
model = GaussianMixture(n_components=2)
```

```
model.fit(X)
```

```
labels = model.predict(X)
```

```
print("Cluster Labels:", labels)
```

OUTPUT:



```
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text | ▶ Run all Copy to Drive

[17]
✓ 0s # Train the model
      model.fit(X)

      # Predict cluster labels
      labels = model.predict(X)

      # Display results
      print("Data Points:\n", X)
      print("Cluster Labels:\n", labels)
      print("Means of Clusters:\n", model.means_)

... Data Points:
[[ 1]
 [ 2]
 [ 3]
 [10]
 [11]
 [12]]
Cluster Labels:
[1 1 1 0 0 0]
Means of Clusters:
[[11.]
 [ 2.]]
```

RESULT

Thus the Expectation Maximization algorithm was successfully implemented for clustering.

Below is the same lab format including ALGORITHM for each experiment in Machine Learning using Python.

EXP NO: 11

Write a program for the task of Credit Score Classification.

AIM

To build a model for credit score classification.

ALGORITHM

1. Import required libraries.
2. Load or create the dataset.
3. Split the dataset into training and testing sets.
4. Train the classifier model.
5. Predict the output for test data.
6. Evaluate the model accuracy.

PROGRAM CODE

```
# Credit Score Classification
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
# Dataset
```

```
data = {
```

```
    'Income':[25000,40000,50000,60000,20000,70000,80000,30000,90000,100000],
```

```
    'Age':[25,35,45,32,23,40,50,28,55,60],
```

```
    'Loan':[5000,10000,15000,20000,3000,25000,30000,7000,35000,40000],
```

```
    'Score':[0,1,1,2,0,2,2,1,2,2] # 0=Low,1=Medium,2=High
```

```
}
```

```
df = pd.DataFrame(data)
```

```
X = df[['Income','Age','Loan']]
```

```
y = df['Score']
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)
```

```
model = DecisionTreeClassifier()
```

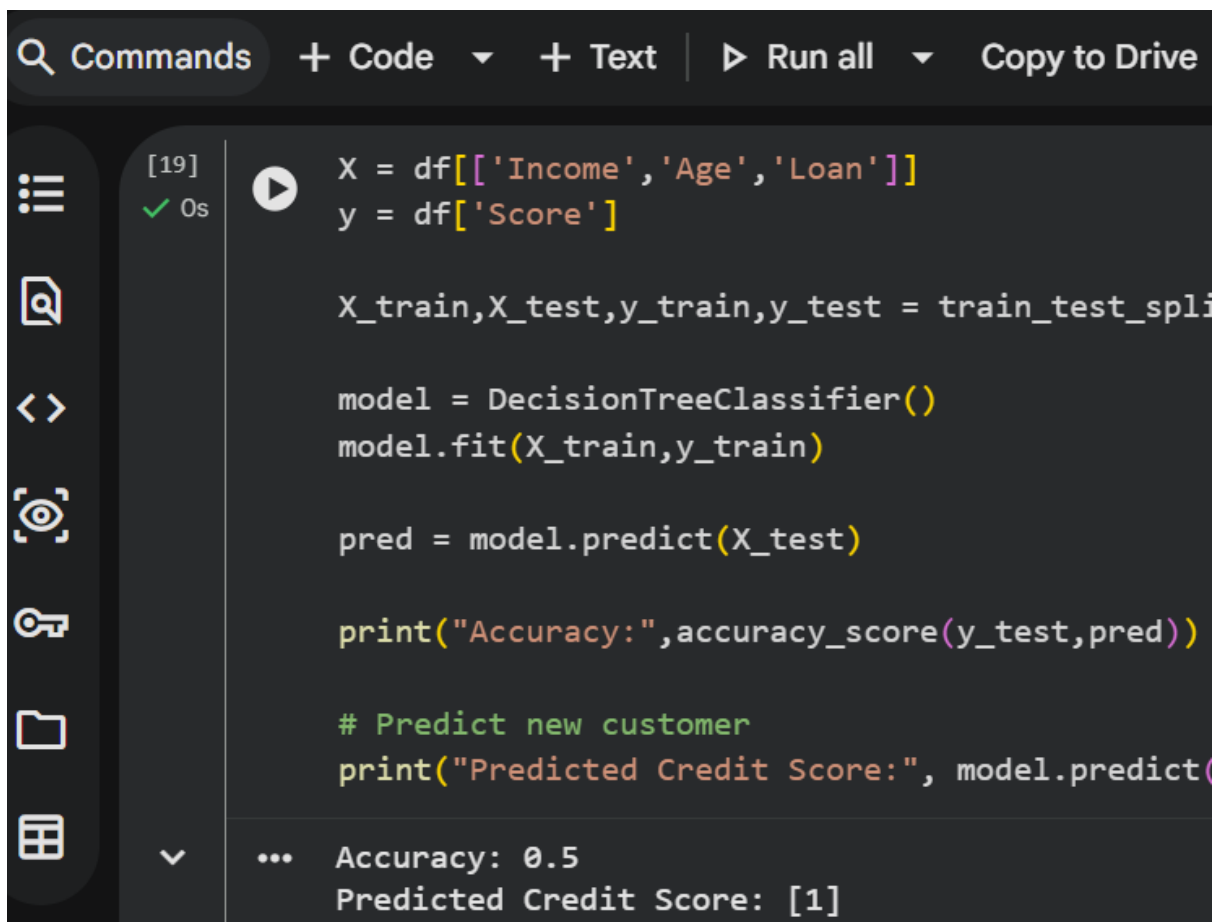
```
model.fit(X_train,y_train)
```

```
pred = model.predict(X_test)
```

```
print("Accuracy:",accuracy_score(y_test,pred))
```

```
# Predict new customer
```

```
print("Predicted Credit Score:", model.predict([[45000,30,12000]])) OUTPUT:
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar contains 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. The left sidebar has icons for a menu, search, expand/collapse, view, key, folder, and a grid. The main area displays a code cell with the following Python code:

```
[19] X = df[['Income','Age','Loan']]
      y = df['Score']

      X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

      model = DecisionTreeClassifier()
      model.fit(X_train,y_train)

      pred = model.predict(X_test)

      print("Accuracy:",accuracy_score(y_test,pred))

      # Predict new customer
      print("Predicted Credit Score:", model.predict([[45000,30,12000]]))
```

Below the code, the output is displayed:

```
... Accuracy: 0.5
      Predicted Credit Score: [1]
```

RESULT

Thus the credit score classification model was successfully implemented.

EXP NO: 12

Implement Iris Flower Classification using KNN.

AIM:

To classify iris flowers using K-Nearest Neighbors algorithm.

ALGORITHM:

1. Import necessary libraries.
2. Load the iris dataset.
3. Split dataset into training and testing sets.
4. Create KNN classifier model.
5. Train the model using training data.
6. Predict the test dataset and evaluate accuracy.

PROGRAM CODE:

```
# Import libraries

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score


# Load dataset

iris = load_iris()
X = iris.data
y = iris.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```

# Create KNN model

model = KNeighborsClassifier(n_neighbors=3)

# Train model

model.fit(X_train, y_train)

# Predict

pred = model.predict(X_test)

# Accuracy

print("Accuracy:", accuracy_score(y_test, pred))

# Predict new flower

print("Predicted Class:", model.predict([[5.1,3.5,1.4,0.2]]))

```

OUTPUT:

```

File Edit View Insert Runtime Tools Help
[21] X = iris.data
      y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Predict
pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, pred))

# Predict new flower
print("Predicted Class:", model.predict([[5.1,3.5,1.4,0.2]]))

... Accuracy: 0.9666666666666667
    Predicted Class: [0]

```

RESULT

Thus iris flower classification using KNN was successfully implemented.

EXP NO: 13

Implement the Car Price Prediction Model using Python.

AIM

To predict car price using regression.

ALGORITHM

1. Import necessary libraries.
2. Prepare dataset with input features and target values.
3. Create regression model.
4. Train the model using training data.
5. Predict the price using test input.
6. Display the predicted output.

PROGRAM CODE

```
# Car Price Prediction using Linear Regression
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error
```

```
# Sample dataset
```

```
data = {  
    'Year':[2015,2016,2017,2018,2019,2020,2021,2014,2013,2022],  
    'Mileage':[50000,40000,30000,20000,15000,10000,8000,60000,70000,5000],  
    'Price':[500000,550000,600000,650000,700000,750000,800000,450000,400000,850000]  
}
```

```
df = pd.DataFrame(data)
```

```
# Features and Target
X = df[['Year','Mileage']]
y = df['Price']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)

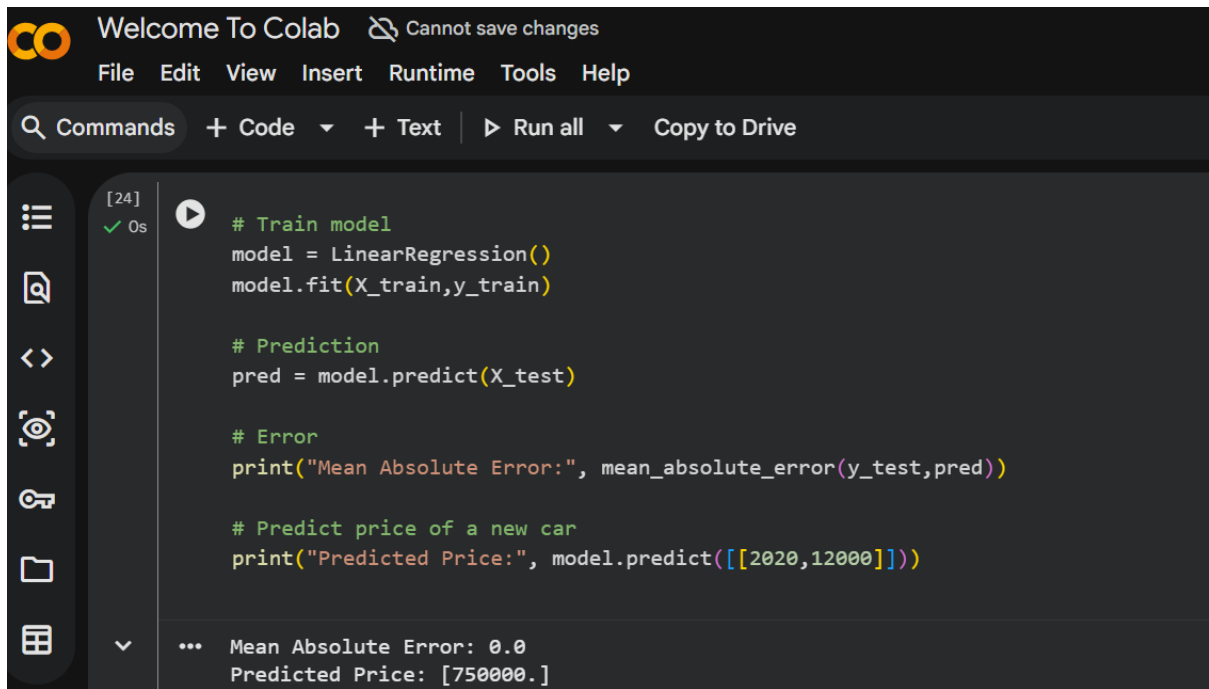
# Train model
model = LinearRegression()
model.fit(X_train,y_train)

# Prediction
pred = model.predict(X_test)

# Error
print("Mean Absolute Error:", mean_absolute_error(y_test,pred))

# Predict price of a new car
print("Predicted Price:", model.predict([[2020,12000]]))
```

OUTPUT:

The image shows a Google Colab notebook interface. At the top, it says "Welcome To Colab" and "Cannot save changes". Below the menu bar (File, Edit, View, Insert, Runtime, Tools, Help), there's a toolbar with "Commands", "+ Code", "+ Text", "Run all", and "Copy to Drive". The notebook contains a code cell with the following Python code:

```
[24] ✓ Os # Train model
      model = LinearRegression()
      model.fit(X_train,y_train)

      # Prediction
      pred = model.predict(X_test)

      # Error
      print("Mean Absolute Error:", mean_absolute_error(y_test,pred))

      # Predict price of a new car
      print("Predicted Price:", model.predict([[2020,12000]]))
```

Below the code, the output is displayed:

```
... Mean Absolute Error: 0.0
      Predicted Price: [750000.]
```

RESULT

Thus the car price prediction model was successfully implemented.

EXP NO: 14

Implement House Price Prediction using appropriate machine learning algorithm.

AIM

To predict house prices using regression.

ALGORITHM

1. Import libraries.
2. Create dataset with house features and prices.
3. Train regression model.
4. Predict house price using trained model.
5. Display predicted result.

PROGRAM CODE

House Price Prediction using Linear Regression

import pandas as pd

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Sample dataset
data = {
    'Area':[1000,1200,1500,1800,2000,2200,2500,2700,3000,3200],
    'Bedrooms':[2,2,3,3,3,4,4,4,5,5],
    'Age':[10,8,6,5,4,3,2,2,1,1],
    'Price':[200000,220000,250000,280000,300000,320000,350000,370000,400000,420000]
}

df = pd.DataFrame(data)

# Features and Target
X = df[['Area','Bedrooms','Age']]
y = df['Price']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train,y_train)

# Prediction
pred = model.predict(X_test)

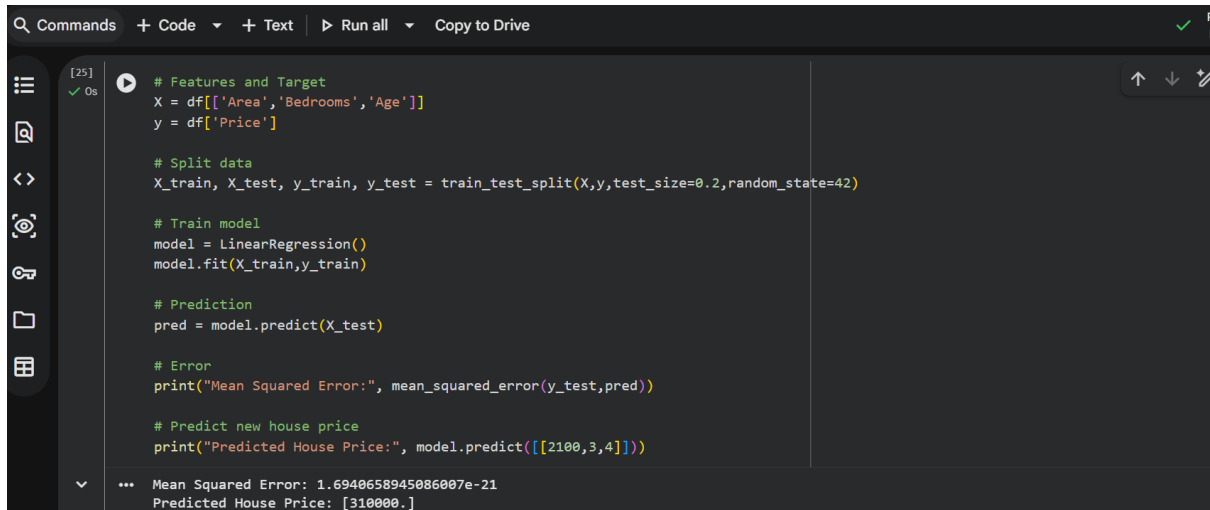
# Error
```

```
print("Mean Squared Error:", mean_squared_error(y_test,pred))
```

```
# Predict new house price
```

```
print("Predicted House Price:", model.predict([[2100,3,4]]))
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes a search icon, 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. The left sidebar contains icons for file management and search. The main area displays a code cell with the following Python code:

```
[25] ✓ 0s
# Features and Target
X = df[['Area', 'Bedrooms', 'Age']]
y = df['Price']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)

# Error
print("Mean Squared Error:", mean_squared_error(y_test, pred))

# Predict new house price
print("Predicted House Price:", model.predict([[2100, 3, 4]]))
```

Below the code cell, the output is displayed:

```
... Mean Squared Error: 1.6940658945086007e-21
Predicted House Price: [310000.]
```

RESULT

Thus the house price prediction model was successfully implemented.

EXP NO: 15

Implement Iris Flower Classification using Naive Bayes classifier.

AIM

To classify iris flowers using Naive Bayes classifier.

ALGORITHM

1. Import libraries.
2. Load iris dataset.
3. Split dataset into training and testing sets.
4. Train Naive Bayes model.
5. Predict output for test dataset.
6. Calculate accuracy.

PROGRAM CODE

```
# Iris Flower Classification using Naive Bayes
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

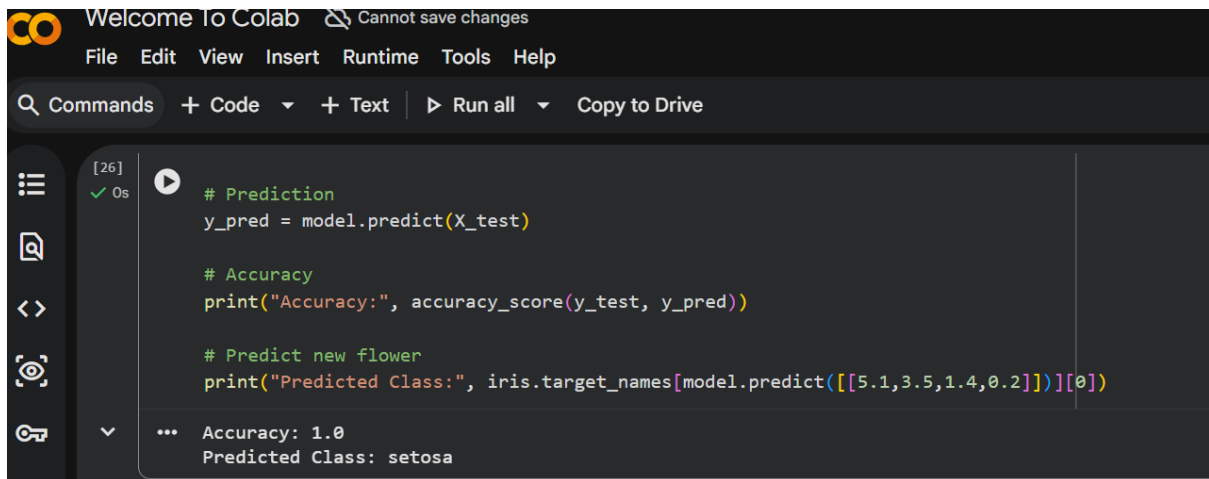
# Create model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Predict new flower
print("Predicted Class:", iris.target_names[model.predict([[5.1, 3.5, 1.4, 0.2]])][0])
```

OUTPUT:



```

Welcome To Colab  Cannot save changes
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all Copy to Drive

[26] ✓ 0s
# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Predict new flower
print("Predicted Class:", iris.target_names[model.predict([[5.1, 3.5, 1.4, 0.2]])[0]])

... Accuracy: 1.0
Predicted Class: setosa

```

RESULT

Thus iris flower classification using Naive Bayes was successfully implemented.

EXP NO: 16

Compare different types of classification algorithms.

AIM

To compare the performance of multiple classification models.

ALGORITHM

1. Import required libraries.
2. Load the dataset.
3. Split the dataset into training and testing data.
4. Train different classification models.
5. Predict outputs using each model.
6. Compare their accuracies.

PROGRAM CODE

```
# Compare Classification Algorithms
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB

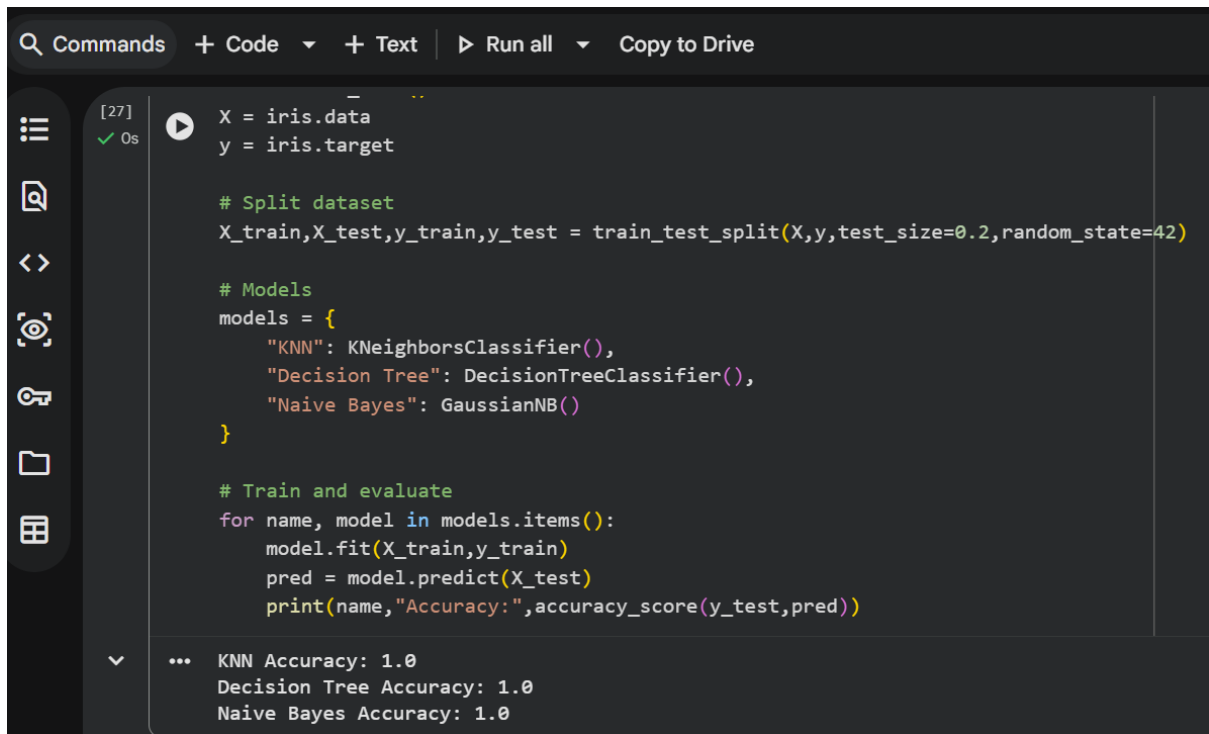
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

# Models
models = {
    "KNN": KNeighborsClassifier(),
    "Decision Tree": DecisionTreeClassifier(),
    "Naive Bayes": GaussianNB()
}

# Train and evaluate
for name, model in models.items():
    model.fit(X_train,y_train)
    pred = model.predict(X_test)
    print(name,"Accuracy:",accuracy_score(y_test,pred))
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface. At the top, there is a toolbar with icons for 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. On the left, a vertical sidebar contains icons for file management and view toggles. The main area displays a code cell with the following Python code:

```
[27] X = iris.data
      y = iris.target

      # Split dataset
      X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

      # Models
      models = {
          "KNN": KNeighborsClassifier(),
          "Decision Tree": DecisionTreeClassifier(),
          "Naive Bayes": GaussianNB()
      }

      # Train and evaluate
      for name, model in models.items():
          model.fit(X_train,y_train)
          pred = model.predict(X_test)
          print(name,"Accuracy:",accuracy_score(y_test,pred))
```

Below the code cell, the output is displayed:

```
... KNN Accuracy: 1.0
     Decision Tree Accuracy: 1.0
     Naive Bayes Accuracy: 1.0
```

RESULT

Thus different classification algorithms were compared successfully.

EXP NO: 17

Implement Mobile Price Prediction using machine learning.

AIM

To predict mobile price using regression.

ALGORITHM

1. Import required libraries.
2. Prepare dataset with mobile features and price.
3. Train regression model.
4. Predict mobile price.
5. Display result.

PROGRAM CODE

```
# Mobile Price Prediction using Decision Tree
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Sample dataset
data = {
    'RAM':[2,3,4,6,8,12,16,3,4,6],
    'Storage':[16,32,64,128,128,256,512,32,64,128],
    'Battery':[3000,3200,3500,4000,4500,5000,6000,3300,3600,4200],
    'Price_Range':[0,0,1,1,2,2,3,0,1,2] # 0=Low,1=Medium,2=High,3=Premium
}

df = pd.DataFrame(data)

# Features and target
X = df[['RAM','Storage','Battery']]
y = df['Price_Range']

# Split dataset
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

# Train model
model = DecisionTreeClassifier()
model.fit(X_train,y_train)

# Predict
pred = model.predict(X_test)

# Accuracy
```

```
print("Accuracy:",accuracy_score(y_test,pred))
```

```
# Predict new mobile price range
```

```
print("Predicted Price Range:",model.predict([[8,128,4500]]))
```

OUTPUT:

```
Welcome To Colab  Cannot save changes
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text | ▶ Run all Copy to Drive

[28] ✓ 0s
x = df[['RAM', 'Storage', 'Battery']]
y = df['Price_Range']

# Split dataset
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

# Train model
model = DecisionTreeClassifier()
model.fit(X_train,y_train)

# Predict
pred = model.predict(X_test)

# Accuracy
print("Accuracy:",accuracy_score(y_test,pred))

# Predict new mobile price range
print("Predicted Price Range:",model.predict([[8,128,4500]]))

... Accuracy: 1.0
Predicted Price Range: [2]
```

RESULT

Thus mobile price prediction was successfully implemented.

EXP NO: 18

Implement Perceptron based IRIS classification.

AIM

To classify the iris dataset using the Perceptron algorithm.

ALGORITHM

1. Import required libraries.
2. Load the iris dataset.
3. Split the dataset into training and testing data.
4. Create the perceptron model.
5. Train the model using training data.

6. Predict the class labels for test data.
7. Evaluate the model accuracy.

PROGRAM CODE

Perceptron based IRIS Classification

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
```

Load dataset

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Train Perceptron model

```
model = Perceptron()
```

```
model.fit(X_train, y_train)
```

Prediction

```
pred = model.predict(X_test)
```

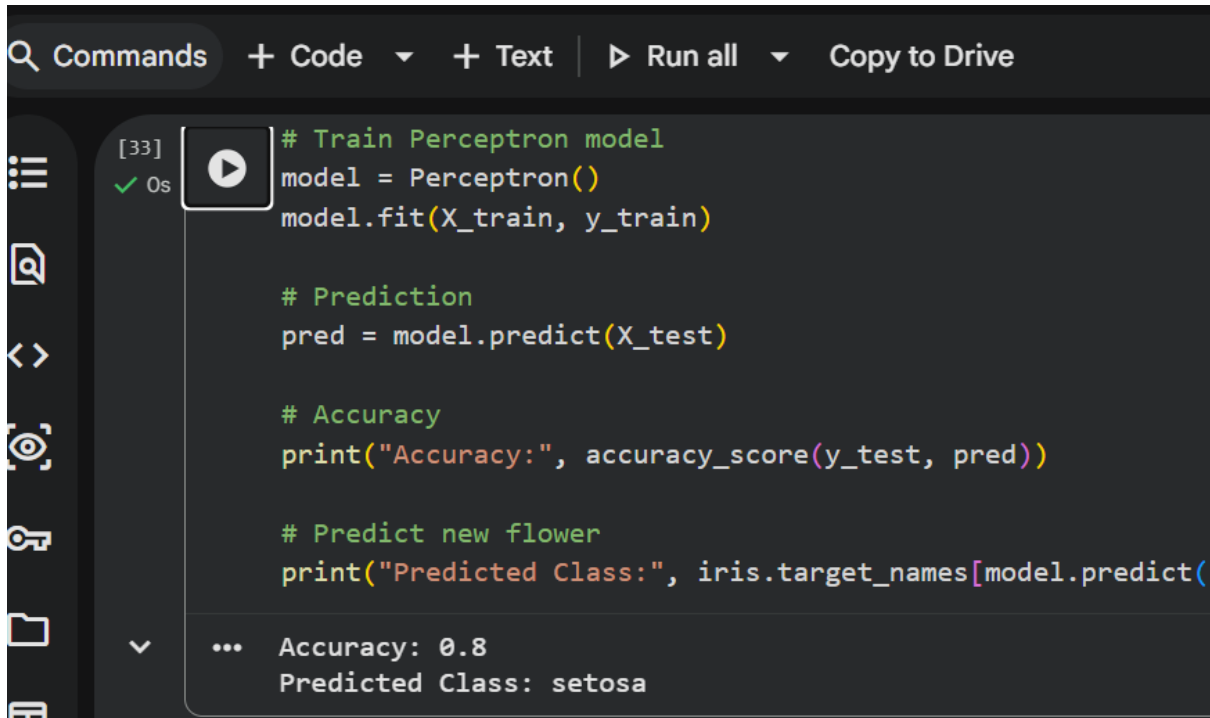
Accuracy

```
print("Accuracy:", accuracy_score(y_test, pred))
```

Predict new flower

```
print("Predicted Class:", iris.target_names[model.predict([[5.1,3.5,1.4,0.2]])][0])
```

OUTPUT:



```
[33] ✓ 0s # Train Perceptron model
      model = Perceptron()
      model.fit(X_train, y_train)

      # Prediction
      pred = model.predict(X_test)

      # Accuracy
      print("Accuracy:", accuracy_score(y_test, pred))

      # Predict new flower
      print("Predicted Class:", iris.target_names[model.predict(
... Accuracy: 0.8
    Predicted Class: setosa
```

RESULT

Thus the iris dataset classification using the Perceptron algorithm was successfully implemented.

EXP NO: 19

Implementation of Naive Bayes classification for Bank Loan prediction.

AIM

To predict bank loan approval using the Naive Bayes classifier.

ALGORITHM

1. Import required libraries.
2. Create a dataset with customer income and loan status.
3. Split the dataset into training and testing sets.
4. Train the Naive Bayes model.
5. Predict loan approval for test data.
6. Display prediction results.

PROGRAM CODE

Bank Loan Prediction using Naive Bayes

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score
```

Sample dataset

```
data = {  
    'Income':[25000,40000,50000,60000,20000,70000,80000,30000,90000,100000],  
    'Credit_Score':[600,650,700,750,580,720,780,640,800,820],  
    'Loan':[5000,10000,15000,20000,3000,25000,30000,7000,35000,40000],  
    'Approved':[0,0,1,1,0,1,1,0,1,1] # 0=No,1=Yes  
}
```

```
df = pd.DataFrame(data)
```

Features and target

```
X = df[['Income','Credit_Score','Loan']]
```

```
y = df['Approved']
```

Split dataset

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

Train model

```
model = GaussianNB()
```

```
model.fit(X_train,y_train)
```

```
# Prediction
```

```
pred = model.predict(X_test)
```

```
# Accuracy
```

```
print("Accuracy:", accuracy_score(y_test,pred))
```

```
# Predict new customer loan approval
```

```
print("Loan Approval Prediction:", model.predict([[45000,680,12000]])) OUTPUT:
```

```
[32] ✓ 0s # Features and target
X = df[['Income', 'Credit_Score', 'Loan']]
y = df['Approved']

# Split dataset
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

# Train model
model = GaussianNB()
model.fit(X_train,y_train)

# Prediction
pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test,pred))

# Predict new customer loan approval
print("Loan Approval Prediction:", model.predict([[45000,680,12000]]))

... Accuracy: 0.5
Loan Approval Prediction: [1]
```

RESULT

Thus bank loan prediction using Naive Bayes classification was successfully implemented.

EXP NO: 20

Implement Future Sales Prediction using a suitable machine learning algorithm.

AIM

To predict future sales using the Linear Regression algorithm.

ALGORITHM

1. Import required libraries.
2. Create dataset containing past sales data.
3. Split dataset into input and output variables.
4. Train the regression model.
5. Predict future sales values.
6. Display predicted sales.

PROGRAM CODE

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Past sales data
X = np.array([[1],[2],[3],[4],[5]])
y = np.array([100,120,140,160,180])

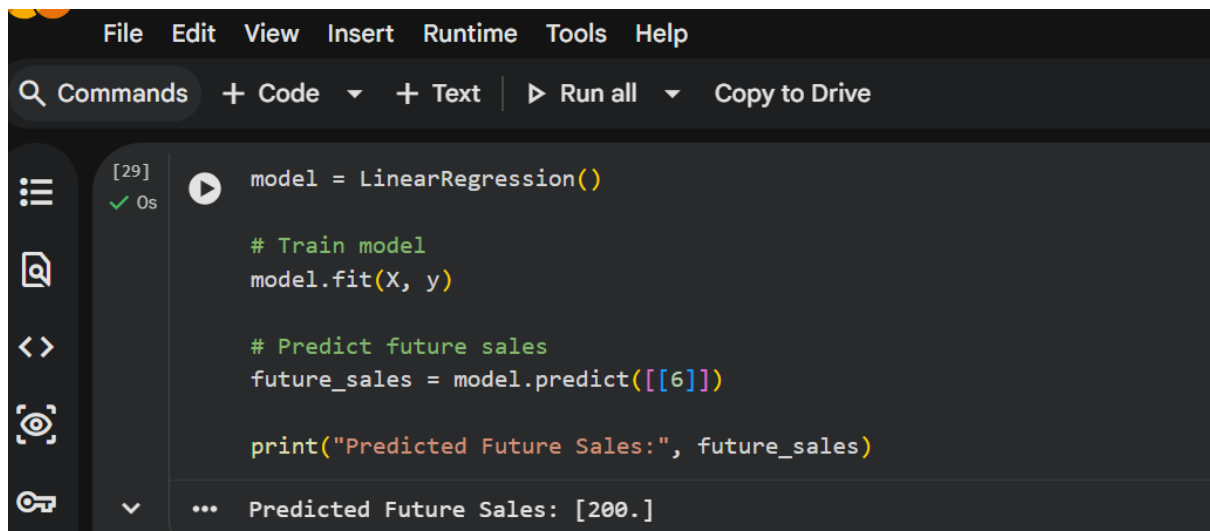
# Create model
model = LinearRegression()

# Train model
model.fit(X, y)

# Predict future sales
future_sales = model.predict([[6]])

print("Predicted Future Sales:", future_sales)
```

OUTPUT:



The screenshot displays a Jupyter Notebook interface with a dark theme. The top menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', 'Run all', and 'Copy to Drive'. The left sidebar contains icons for a menu, search, navigation, and a key. The main area shows a code cell with the following Python code:

```
[29]
✓ Os
model = LinearRegression()

# Train model
model.fit(X, y)

# Predict future sales
future_sales = model.predict([[6]])

print("Predicted Future Sales:", future_sales)
```

Below the code cell, the output is displayed: '... Predicted Future Sales: [200.]'.

RESULT

Thus future sales prediction using machine learning was successfully implemented.